Apriori Algorithm:

The **Apriori algorithm** is a classic algorithm used in **data mining** for **association rule learning**. It aims to find associations between different items in large datasets, often used for market basket analysis. The algorithm generates frequent itemsets from transactional data and uses them to create rules that can help predict the occurrence of items based on the presence of other items.

How it works:

1. **Generate Frequent Itemsets:** Find itemsets that occur frequently together in transactions. These itemsets must meet a **minimum support** threshold.

2. **Generate Association Rules:** For each frequent itemset, generate rules that show the relationships between the items. These rules must meet both **minimum confidence** and **minimum lift** thresholds.

Key Parameters:

- **Support**: The proportion of transactions in the dataset that contain a particular itemset.

$$\text{Support}(A) = \frac{\text{Transactions containing itemset A}}{\text{Total transactions}}$$

- **Confidence**: The likelihood that an item B will be bought when item A is bought.

$$\text{Confidence}(A \to B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

- **Lift**: The ratio of observed support to expected support if A and B were independent.

$$\text{Lift}(A \to B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$$

Steps of the Apriori Algorithm:

1. **Identify Frequent 1-itemsets**: Find all individual items that meet the minimum support threshold.

2. **Generate Candidate 2-itemsets**: From frequent 1-itemsets, generate potential 2-itemsets.

3. **Count support for Candidate 2-itemsets**: Calculate the support of the candidate 2-itemsets.

4. **Repeat**: Continue generating candidate itemsets of increasing size (3-itemsets, 4-itemsets, etc.), pruning those that don't meet the support threshold.
5. **Generate Rules**: For each frequent itemset, generate association rules that satisfy the minimum confidence threshold.

Example Problem:

Consider a retail store with the following transaction data:

Transaction ID	**Items Purchased**
T1	{Bread, Milk}
T2	{Bread, Diaper, Beer, Milk}
T3	{Milk, Diaper, Beer, Cola}
T4	{Bread, Milk, Diaper, Cola}
T5	{Bread, Milk, Diaper, Beer}

Let's set the **minimum support** as 60% and the **minimum confidence** as 80%.

Step 1: Find Frequent 1-itemsets

- Bread: Appears in 4 transactions → Support = 4/5 = 80%
- Milk: Appears in 4 transactions → Support = 4/5 = 80%
- Diaper: Appears in 4 transactions → Support = 4/5 = 80%
- Beer: Appears in 3 transactions → Support = 3/5 = 60%
- Cola: Appears in 2 transactions → Support = 2/5 = 40%

Frequent 1-itemsets (support >= 60%):

- {Bread}, {Milk}, {Diaper}, {Beer}

Step 2: Generate Candidate 2-itemsets

Now, generate combinations of 2-itemsets:

- {Bread, Milk}: Appears in 3 transactions → Support = 3/5 = 60%
- {Bread, Diaper}: Appears in 3 transactions → Support = 3/5 = 60%

- {Bread, Beer}: Appears in 2 transactions → Support = 2/5 = 40%
- {Milk, Diaper}: Appears in 3 transactions → Support = 3/5 = 60%
- {Milk, Beer}: Appears in 2 transactions → Support = 2/5 = 40%
- {Diaper, Beer}: Appears in 3 transactions → Support = 3/5 = 60%

Frequent 2-itemsets (support >= 60%):

- {Bread, Milk}, {Bread, Diaper}, {Milk, Diaper}, {Diaper, Beer}

Step 3: Generate Candidate 3-itemsets

Now, generate combinations of 3-itemsets:

- {Bread, Milk, Diaper}: Appears in 2 transactions → Support = 2/5 = 40%
- {Bread, Milk, Beer}: Appears in 2 transactions → Support = 2/5 = 40%
- {Milk, Diaper, Beer}: Appears in 2 transactions → Support = 2/5 = 40%

No frequent 3-itemsets (support >= 60%).

Step 4: Generate Rules

For frequent 2-itemsets, generate rules that satisfy the confidence threshold of 80%. For example:

- {Bread} → {Milk}: Confidence = Support({Bread, Milk}) / Support({Bread}) = 60% / 80% = 75% (Not frequent enough)
- {Milk} → {Bread}: Confidence = Support({Bread, Milk}) / Support({Milk}) = 60% / 80% = 75% (Not frequent enough)

Resulting Association Rules:

- {Bread, Diaper} → {Milk} with confidence 80%
- {Milk, Diaper} → {Bread} with confidence 80%

This means, for example, that customers who buy **Bread** and **Diaper** will likely also buy **Milk** with 80% confidence.

Conclusion:

The Apriori algorithm identifies frequently bought items and provides insights into how items are related, which can help businesses improve product placement, marketing, and inventory decisions